Student Placement Prediction Using Machine Learning (SVM)

Submitted by

BHAVANI KAMERA
(1DS22IS145 | B.E. – Artificial Intelligence and Data Science |)
Chaitanya Bharathi Institute of Technology, Hyderabad

Under the guidance of

Vinod Kumar

Scientist 'F'

16 December to 16 January

Centre for Airborne Systems (CABS), DRDO

Near Wind Tunnel Road, Belur, Yemalur, Post, Bengaluru, Karnataka 560037

ACKNOWLEDGEMENT

I am profoundly grateful to the ***Centre for Airborne Systems (CABS), DRDO***, for granting me the opportunity to undertake my internship at this distinguished defense research establishment. The exposure I received here has been invaluable in expanding both my technical knowledge and professional outlook.

I would like to sincerely thank ***Mr. Maheswaran, Scientist F***, for his continuous encouragement and for fostering an environment that motivated me to push my boundaries during the course of this internship. His guidance provided me with the confidence to approach challenges with greater clarity.

My deepest appreciation goes to my mentor, ***Mr. Vinod Kumar, Scientist F***, whose mentorship was pivotal to my learning journey. He not only helped me navigate the intricate concepts of Deep Learning but also directed me towards the right resources and areas of focus. His steady guidance and practical insights ensured that I could approach my project with a clear sense of direction.

I would also like to acknowledge the contributions of the technical teams and staff at CABS, whose cooperation and support at different stages of my work greatly enriched my learning experience.

As part of this internship, I had the opportunity to work on a **machine learning–based Student Placement Prediction system**. Leveraging **Python and Scikit-learn,** I gained practical experience in building, training, and evaluating supervised learning models designed for accurate placement classification. This project gave me a deeper appreciation of how artificial intelligence and data science techniques can be harnessed to develop intelligent, data-driven solutions for real-world educational and employability applications.

Overall, this internship has been an immensely rewarding experience that has contributed significantly to both my professional growth and technical skill set. I am truly thankful to everyone who made this journey possible.

Bhavani kamera

ABSTRACT

This internship at ***CABS, DRDO*** centered on applying machine learning techniques to a real-world student placement prediction problem. The primary objective was to study and implement supervised learning models capable of accurately classifying students as placed or not placed based on academic performance and skill-based attributes, even in scenarios where student profiles exhibit overlapping characteristics and non-linear patterns commonly found in real educational data.

The focus of the project was on developing a Student Placement Prediction model using a Soft-Margin **Linear Support Vector Machine (SVM)**, a supervised learning algorithm well suited for binary classification tasks. Unlike conventional threshold-based or rule-driven approaches, the SVM learns an optimal decision boundary by maximizing the margin between placed and non-placed students while allowing controlled misclassifications. This enables the model to effectively handle overlapping academic and skill-based data, where student performance characteristics may not be perfectly separable. The linear nature of the model also provides interpretability, allowing key placement-influencing features to be analyzed and understood within an educational decision-making context.

The model was implemented using **Python** and the **Scikit-learn library**, forming a complete machine learning pipeline for student placement prediction. The system was trained and evaluated on a real-world **student placement dataset containing academic and skill-based attributes**. Model performance was assessed using standard classification metrics, including accuracy, precision, recall, F1-score, and confusion matrix analysis, along with cross-validation to ensure **robustness and generalization** across different data splits.

Through this work, I gained a deeper understanding of how machine learning models, particularly **Support Vector Machines**, can outperform traditional rule-based approaches in predictive analytics tasks. This project highlights the importance of applying well-suited supervised learning algorithms like SVM in real-world educational applications, offering robust and reliable prediction capabilities that support data-driven decision-making in student placement and employability analysis.

TABLE OF CONTENT

TITLE	*PAGE NO*
Acknowledgement	2
Abstract	3
About the Organization	5
1. Machine learning concepts	
1.1 Linear Regression	6
1.2 Logistic Regression	6
1.3 Bias and Variance	7
1.4 Support vector machine	9
1.5 K-Means Clustering	11
1.6 K – Nearest Neighbors	13
2. Deep Learning Concepts	
2.1 Perceptron	16
2.2 Multilayer Perceptron	18
2.3 Activation Functions	20
3.Project Description and Methodology	23
4.Code and Implementation	26
5.Conclusion	28

About the Organization: Centre for Airborne Systems (CABS), DRDO, Bangalore

The Centre for Airborne Systems (CABS) is a premier laboratory under the Defence Research and Development Organization (DRDO), headquarters in Bengaluru, Karnataka. Established in 1991, CABS serves as the nodal agency for the development of advanced airborne surveillance systems for India's defense forces. The organization plays a critical role in designing, developing, and integrating airborne electronic systems that enhance the nation's military capabilities.

Vision and Mission

- Vision: To meet the technological challenges of airborne surveillance systems.
- Mission: To develop key technologies and infrastructure for building efficient and cost-effective airborne surveillance systems.

Certifications and Recognitions :

CABS has been accredited with the prestigious Aerospace Standard (AS9100D) certification by the International Aerospace Quality Group (IAQG), reflecting its commitment to quality and excellence in aerospace engineering. Additionally, it has received numerous awards within DRDO, including performance excellence awards and recognition for its scientists' contributions.

CABS continues to be a cornerstone of India's defense R&D efforts, leveraging cutting-edge technology to ensure national security.

INTRODUCTION

1. What is Machine Learning?

Machine Learning (ML) is a subset of ***Artificial Intelligence (AI)*** that enables systems to ***learn from data*** and ***make predictions or decisions*** without being explicitly programmed.

Types of Machine Learning :-

ML algorithms can be broadly categorized into the following types:

1. *Supervised Learning*

Data: Labeled (input → output)

Goal: Learn a mapping function from input to output Used for: Classification, Regression

2. *Unsupervised Learning*

Data: Unlabeled (no output)

Goal: Discover hidden structure or patterns Used for: Clustering, Dimensionality Reduction

3. *Semi-Supervised Learning*

Mix of labeled and unlabeled data

Cost-effective in domains where labels are expensive (e.g., medical imaging)

4. *Reinforcement Learning (RL)*

Agent learns by interacting with environment

Learns a policy based on ***reward and punishment***

1.1 Linear Regression

Definition: Linear Regression is a supervised machine learning and statistical technique used to model the relationship between a dependent variable (y) and an independent variable (X) under the assumption of linearity. It aims to find the best-fitting straight line that minimizes the error between actual and predicted values. ***Equation of Linear Regression:***

For simple linear regression (with one feature), the model is defined as:

$$y = wx + b$$

Where:

- y is the predicted output,
- x is the input feature,

- w is the weight (slope of the line),
- b is the bias (intercept).

In multiple linear regression (with multiple features):

$y = w_1x_1 + w_2x_2 + ... + w_nx_n + b$

Or in vector form: $y = w^Tx + b$

Error, Cost Function, and Sum of Squares

For any data point X_k:
Residual (error) = (y_true − y_pred)
Residual Sum of Squares (RSS):
RSS = Σ (y_true − y_pred)²
Explained Sum of Squares (ESS): Variability explained by the model.

Total Sum of Squares (TSS): Total variability in the actual data.

Relationship: TSS = ESS + RSS

Coefficient of Determination (R^2)

The coefficient of determination measures the proportion of variance in y that can be explained by X.
$R^2 = 1 - (RSS / TSS)$

Interpretation:
R^2 close to 1 indicates a strong model fit.
R^2 close to 0 indicates a weak model fit.

Adjusted R^2

R^2 always increases when new predictors are added, even if they are irrelevant. Adjusted R^2 penalizes extra predictors.
Adjusted $R^2 = 1 - [(1 - R^2) \times (N - 1) / (N - P - 1)]$

where N is the sample size and P is the number of predictors.

1.2 Logistic Regression

Definition: Logistic Regression is a supervised learning algorithm used for classification problems, where the dependent variable is categorical (e.g., 0/1, yes/no). It uses the sigmoid function to map inputs into probabilities between 0 and 1.

Logistic Function

The hypothesis is given by the ***sigmoid function***: $h(x) = 1 / (1 + e^{(-z)})$

where $z = c + mX$. If $h(x) \geq 0.5 \rightarrow$ predict class 1, otherwise class 0.

Loss Function

The model is trained using ***Log Loss (Cross-Entropy)***:

$J(\theta) = -(1/n) \Sigma [y_i \log(h(x_i)) + (1 - y_i) \log(1 - h(x_i))]$

Optimization

Parameters are optimized using ***Gradient Descent***: $\theta_new = \theta_old - \eta \times \partial J/\partial\theta$

1.3 Bias and Variance

Definition of Bias Bias refers to the error introduced when a model makes simplifying assumptions to approximate a real-world problem. It is the difference between the expected prediction of the model and the true underlying value.

High Bias (Underfitting): A model with high bias is too simple to capture the underlying patterns in the data.

Symptoms: High training error and high test error. Example: Approximating a curved relationship with a straight line.

Definition of Variance

Variance refers to the sensitivity of a model to small fluctuations in the training data. It measures how much the model's predictions would vary if trained on different subsets of the data.

High Variance (Overfitting): A model with high variance is overly complex and learns both patterns and noise in the training data.

Symptoms: Very low training error but high test error. Example: A deep decision tree that memorizes training data instead of generalizing.

Bias–Variance Trade-off

The bias–variance trade-off is a fundamental concept in supervised machine learning. It describes the balance required between bias and variance to achieve good generalization.

Simple Models: High bias, low variance (consistent but inaccurate).

Complex Models: Low bias, high variance (accurate on training but poor on unseen data).

Goal: Achieve an optimal balance where total error is minimized.

The total error of a model can be expressed as:
Total Error = Bias² + Variance + Irreducible Error

Irreducible Error: Noise inherent in the data that cannot be eliminated.

Strategies for Managing the Trade-off

To Reduce Bias (combat underfitting):

• Increase model complexity
• Use more relevant features
• Reduce regularization

To Reduce Variance (combat overfitting):

• Simplify the model
• Increase training data size
• Apply regularization (L1/L2, dropout)
• Use ensemble methods (e.g., Random Forests, Bagging)

1.4 Support Vector Machine

Definition
Support Vector Machine (SVM) is a powerful supervised machine learning algorithm mainly used for classification and regression tasks. It works by finding the optimal hyperplane that best separates data points of different classes in a high-dimensional space. The data points closest to this hyperplane are called ***support vectors***, and they are critical in defining the decision boundary.

Why SVM is used?

- Effective in high-dimensional spaces. Works well even when the number of features is
- greater than the number of samples. Robust to overfitting, especially in high-dimensional
- datasets. Versatile: different kernel functions can be applied to model complex, non-linear
- decision boundaries.

Mathematical Formulation
For a binary classification problem, given a dataset with feature vectors xi and labels $yi \in -1, +1$, SVM tries to find a hyperplane:

$$w \cdot x + b = 0$$

Where w is the weight vector, b is the bias, and x is the input feature vector. The objective of the Support Vector Machine is to maximize the margin, which is the distance between the separating hyperplane and the nearest data points, while still correctly classifying as many training examples as possible.

The optimization goal can be expressed as:

$$\min \frac{1}{2}\|w\|^2 \quad \text{subject to} \quad y_i\left(w \cdot x_i + b\right) \geq 1$$

this ensures maximum margin classification.

Types of SVMs

Linear SVM

- Used when data is linearly separable.
- Finds a straight hyperplane to separate classes.

Non-Linear SVM

- Used when data cannot be separated by a straight line.
- Uses the ***kernel trick*** to map data into a higher-dimensional space where separation is possible.

Common Kernel Functions

Linear Kernel:

$$K(x_i, x_j) = x_i \cdot x_j$$

- The simplest kernel, used when the data is linearly separable.
- Equivalent to standard linear classification.

Polynomial Kernel

Allows learning of non-linear relationships.

$$K(x_i, x_j) = (x_i \cdot x_j + c)^d$$

Parameters:
c: Trade-off parameter, controls flexibility.
d: Degree of the polynomial, controls the complexity of the decision boundary.

RBF Kernel

This kernel is similar to neural networks since it uses the hyperbolic tangent (tanh) activation function. It is useful in certain cases but less commonly applied than the RBF and polynomial kernels.

$$K(x_i, x_j) = \tanh\big(\alpha(x_i \cdot x_j) + c\big)$$

Parameters:
α and c control the kernel shape.

Sigmoid Kernel

The Sigmoid kernel is defined as:

$$K(x_i, x_j) = \tanh\big(\alpha(x_i \cdot x_j) + c\big)$$

It is similar to neural networks because it uses the hyperbolic tangent (tanh) activation function. This kernel can model certain non-linear relationships, though it is less commonly used compared to RBF and polynomial kernels. The parameters α and c control the shape of the kernel.

Applications of SVM

- Text classification (e.g., spam detection, sentiment analysis)
- Image recognition and classification
- Bioinformatics (e.g., cancer detection using gene expression data)
- Handwriting recognition
- Face detection

1.5 K-Means Clustering

Definition K-Means is a widely used ***unsupervised machine learning algorithm*** for partitioning a dataset into KKK clusters based on feature similarity. It minimizes intra-cluster variance while maximizing inter-cluster separation, making it one of the most popular clustering techniques in data mining and pattern recognition.

Working Principle

The algorithm works iteratively to assign data points to clusters and refine cluster centroids:

- ***Initialization***: Select KKK initial centroids (randomly or using K-Means++).
- ***Assignment Step***: Assign each data point to the cluster whose centroid is nearest (using a distance metric, usually Euclidean distance).
- ***Update Step***: Recalculate each centroid as the mean of all points in its cluster.
- ***Convergence***: Repeat steps 2 and 3 until centroids stabilize or the change in the cost function becomes negligible.

Mathematical Formulation

The goal of K-Means is to minimize the ***objective function (within-cluster sum of squares):***

$$J = \sum_{i=1}^{K} \sum_{x \in C_i} \|x - \mu_i\|^2$$

K - represents the number of clusters.
Ci - denotes the set of points belonging to cluster i.
μ_i - is the centroid of cluster i.
$\| x - \mu i \| 2$ - represents the squared Euclidean distance between a data point x and the centroid μi.

Key Characteristics

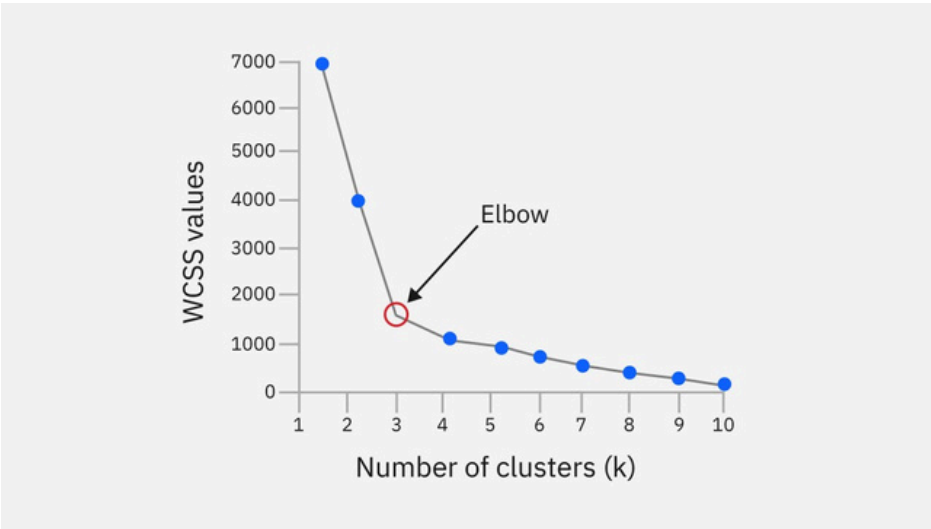
K-Means seeks compact and spherical clusters and is computationally efficient with a time complexity of approximately O(n · K · t), where nnn is the number of data points and ttt is the number of iterations. It also scales well to large datasets.

Applications

K-Means is widely used in practice, such as in customer segmentation in marketing, image compression by reducing the color space, document and text clustering, and anomaly detection in networks and systems.

Limitations

The algorithm requires pre-specifying the number of clusters K, is sensitive to initial centroid placement, and may converge to local minima. Moreover, it assumes clusters are spherical and of similar size, which may not hold true for complex data distributions.



1.6 K-Nearest Neighbors (KNN) Algorithm

K-Nearest Neighbors (KNN) is a simple, non-parametric, lazy learning algorithm used for both ***classification*** and ***regression*** tasks. It's considered "non-parametric" because it makes no explicit assumptions about the underlying data distribution. It's "lazy" because it does not build a generalized model during the training phase; instead, it memorizes the training dataset and performs computations only when a prediction is requested.

How it Works:

The core idea behind KNN is that similar data points tend to exist in close proximity. When predicting the class or value for a new, unseen

data point, KNN looks at its 'K' nearest neighbors in the training dataset.

1. ***Choose K:*** Select the number of neighbors (K) to consider. This is a crucial hyperparameter.
2. ***Calculate Distance:*** For a new data point, calculate its distance to all training data points. Common distance metrics include:
   - ***Euclidean Distance:*** Most common for continuous features. $d(p,q)=\sum_{i=1}^{n}(q_i-p_i)$
   - ***Manhattan Distance:*** Also known as L1 distance. $d(p,q)=\sum_{i=1}^{n}|q_i-p_i|$
   - ***Minkowski Distance:*** A generalization of Euclidean and Manhattan distances.
3. ***Identify K Nearest Neighbors:*** Select the K data points from the training set that have the smallest distances to the new data point.
4. ***Prediction:***
   - ***For Classification:*** The new data point is assigned the class that is most frequent among its K nearest neighbors (majority vote).
   - ***For Regression:*** The new data point's value is the average (or weighted average) of the values of its K nearest neighbors.

Key Components:

- ***K Value:*** The number of neighbors to consider. A small K can lead to noise sensitivity (high variance), while a large K can blur class boundaries (high bias).
- ***Distance Metric:*** The function used to measure the similarity between data point
- ***Voting Mechanism (for Classification):*** Majority vote is standard, but weighted voting (where closer neighbors have more influence) can also be used.

Advantages:

- ***Simple and Intuitive:*** Easy to understand and implement.
- ***No Training Phase (Lazy Learning):*** No explicit model building, making it quick to "train."
- ***Non-parametric:*** Makes no assumptions about data distribution, making it flexible for various data types.
- ***Handles Multi-class Problems Naturally:*** Easily extends to more than two classes.

Limitations:

- ***Computationally Expensive (at prediction time):*** Requires calculating distances to all training points for each new prediction, which can be slow for large datasets.
- ***Sensitive to the Curse of Dimensionality:*** Performance degrades significantly with a high number of features, as distances become less meaningful.

- ***Sensitive to Irrelevant Features:*** Features that don't contribute to the prediction can negatively impact performance.
- ***Requires Feature Scaling:*** Features with larger ranges can disproportionately influence distance calculations, so normalization/standardization is often necessary.
- ***Optimal K Value:*** Choosing the right K can be challenging and often requires cross-validation.

2.Introduction to Deep Learning

What is Deep Learning?

Deep Learning is a subset of ***Machine Learning*** that uses algorithms known as ***artificial neural networks***, inspired by the human brain, to learn from large amounts of data. It is called "deep" because these neural networks have multiple layers — allowing them to model complex, high-level patterns.

How It Works

A typical deep learning model consists of layers of ***neurons***, where:

- Each neuron receives inputs, applies a ***weighted sum***, and passes the result through an ***activation function*** (like ReLU or Sigmoid).
- The model adjusts weights through ***backpropagation*** using a ***loss function*** and ***gradient descent*** to reduce prediction errors.

Types of Deep Learning Architectures

Architecture	Used For
DNN (Deep Neural Network)	Basic classification/regression
CNN (Convolutional Neural Network)	Image recognition, object detection
RNN (Recurrent Neural Network)	Sequence data like text or time series
LSTM/GRU	Long-term dependencies in sequences
Transformer	Language modeling (e.g., ChatGPT)
Autoencoder	Dimensionality reduction, anomaly detection
GAN (Generative Adversarial Network)	Data generation (images, videos, etc.)

Applications of Deep Learning :-

- ***Computer Vision***: Face recognition,

self-driving cars ***Natural Language Processing***: Chatbots, machine translation ***Speech Recognition***: Voice assistants like Siri or Alexa

‘

Machine Learning	***Deep Learning***
Teaches machines using algorithms on data features.	Uses neural networks with multiple hidden layers.
Works with small/medium datasets.	Requires large amounts of data.
Manual feature extraction is needed.	Features are learned automatically.
Less computationally intensive.	Requires high computing power (GPUs/TPUs).
Faster to train.	Longer training time.
Easier to interpret results.	Works as a “black box” (less interpretable).
Good for simpler tasks.	Higher accuracy in complex tasks.
Spam filtering, fraud detection, recommendations.	Image recognition, NLP, speech processing.

2.1 Perceptron

What is a Perceptron?

The ***Perceptron*** is the simplest type of artificial neural network model, introduced by Frank Rosenblatt in 1958. It is a ***binary linear classifier*** that decides whether an input, represented by a vector of numbers, belongs to one class or another.

Mathematically, the perceptron computes a ***weighted sum*** of the input features and passes it through an ***activation function*** (commonly the step function) to produce an output.

Formula:

$$y = f\left(\sum_{i=1}^{n} w_i x_i + b\right)$$

xi: input features

w: weights assigned to each input

b: bias term

$f(\cdot)$: activation function (e.g., step function

y: output (either 0 or 1 in binary classification)

The perceptron is the foundation of modern neural networks. While limited in capability (it can only solve linearly separable problems), it paved the way for the development of ***multi-layer perceptrons (MLPs)*** and ***deep learning***.

How does Perceptron work? A weight is assigned to each input node of a perceptron, indicating

the importance of that
input in determining the output. The Perceptron's output is calculated as a weighted sum of the inputs, which is then passed through an activation function to decide whether the Perceptron will fire.
The weighted sum is computed as:

$$z = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n = X^T W$$

The step function compares this weighted sum to a threshold. If the input is larger than the threshold value, the output is 1; otherwise, it's 0. This is the most common activation function used in Perceptrons are represented by the Heaviside step function:

$$h(z) = \begin{cases} 0, & \text{if } z < \text{Threshold} \\ 1, & \text{if } z \geq \text{Threshold} \end{cases}$$

A perceptron consists of a single layer of Threshold Logic Units (TLU), with each TLU fully connected to all input nodes.

In a fully connected layer, also known as a dense layer, all neurons in one layer are connected to every neuron in the previous layer.
The output of the fully connected layer is computed as:

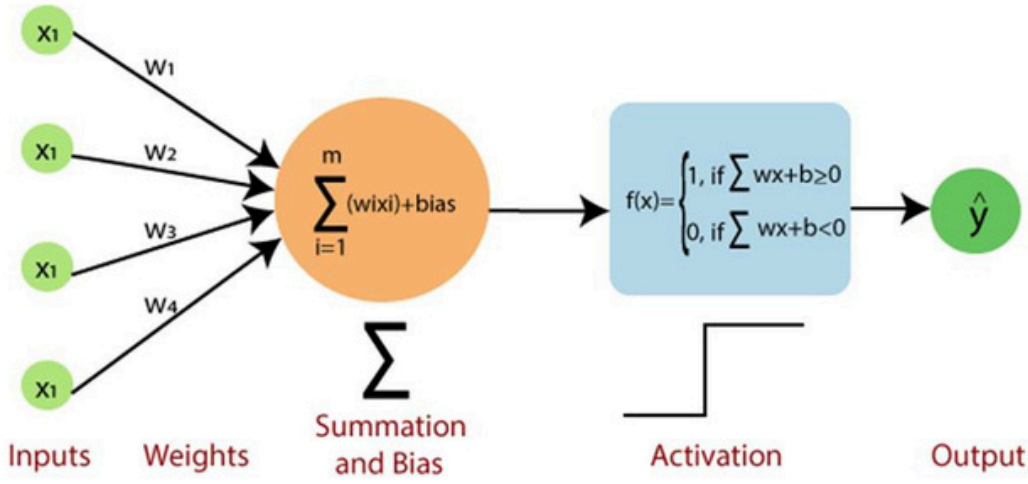
$$f_{W,b}(X) = h(XW + b)$$

where X is the input W is the weight for each inputs neurons andbis the bias and h is the step function.
During training, the Perceptron's weights are adjusted to minimizethe difference between the predicted output and the actual output. This is achieved using supervised learning algorithms like the delta rule or the Perceptron learning rule.
The weight update formula is:

$$w_{i,j} = w_{i,j} + \eta(y_j - \hat{y}_j)x_i$$

Where:

$w_{i,j}$ is the weight between the ith input and jth output neuron,
xi is the ith input value,
yj is the actual value, and is the predicted value,
η is the learning rate, controlling how much the weights are adjusted.
This process enables the perceptron to learn from data and improve its prediction accuracy over time.



Types of Perceptrons :-

Type	Description
Single-layer Perceptron	Can solve only linearly separable problems
Multi-layer Perceptron (MLP)	Has one or more hidden layers, can model complex functions

Limitations of Perceptron

- Only works with ***linearly separable data***
  Example: cannot solve XOR problem.
- Fixed output range (0 or 1)
- Step function is ***not differentiable***, making it unsuitable for gradient-based optimization

These limitations were overcome by ***Multi-Layer Perceptrons (MLPs).***

2.2 Multilayer Perceptron (MLP)

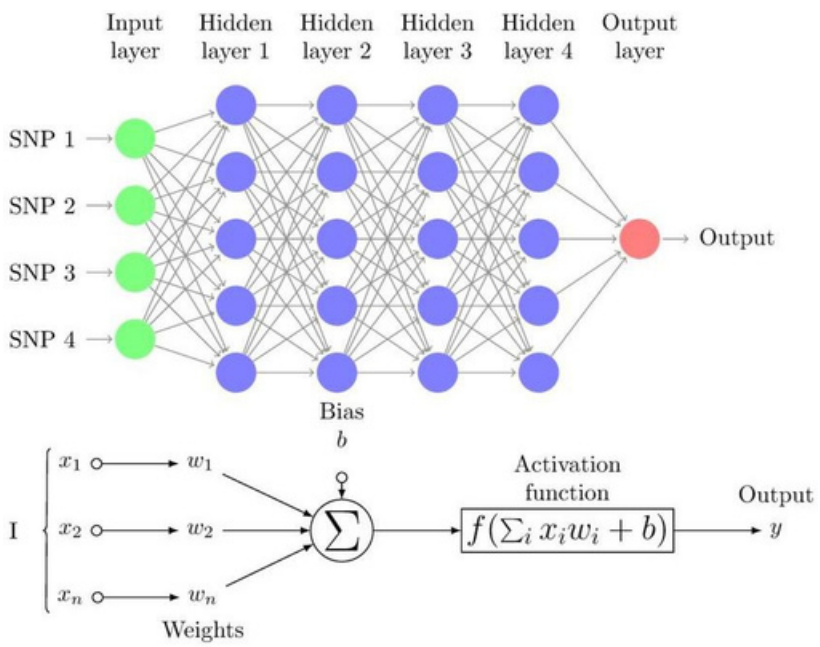
What is a Multilayer Perceptron (MLP)?

A ***Multilayer Perceptron (MLP)*** is a type of ***feedforward artificial neural network*** that consists of ***three or more layers of neurons***:

- Input Layer
- One or more Hidden Layers
- Output Layer

MLPs can model complex, non-linear decision boundaries

Architecture of an MLP :-



Mathematical Foundation

Forward Propagation

For a single hidden layer:

Given:

- Input: x
- Weights: $W[1], W[2]$
- Biases: $b[1], b[2]$
- Activation: $a[1], a[2]$
- Activation function: $f(\cdot)$

The computation steps:

1. Hidden layer:

$$z[1] = W[1]x + b[1]$$
$$a[1] = f(z[1])$$

2. Output layer:

$$z[2] = W[2]a[1] + b[2]$$

$$a[2] = y = f(z[2])$$

2.3 Activation Functions in Neural Networks

What are Activation Functions?

Activation functionsare mathematical functions applied to the output of neurons in a neural network. They introduce ***non-linearity*** into the model, enabling the network to learn and represent complex patterns. Without activation functions, even multi-layer networks would behave like a simple linear regression model, limiting their capability.

Why Do We Need Activation Functions?

Activation functions are crucial because they allow neural networks to model complex, non-linear relationships between inputs and outputs. Without them:

- The network could only solve linearly separable problems.
- Deep networks would essentially collapse into a single-layer linear model.

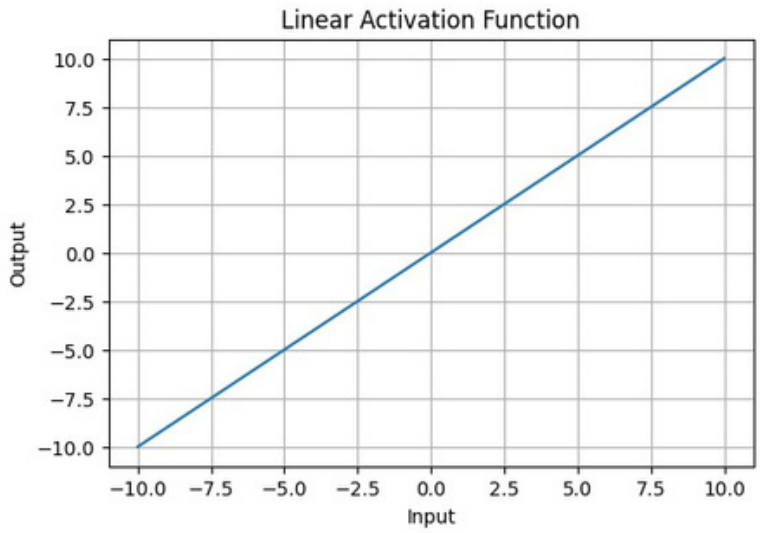
They also play an important role in backpropagation, providing the gradients necessary for updating weights and biases during training.

Types of Activation Functions in Deep Learning

1. Linear Activation Function

Linear Activation Function resembles straight line define by y=x. No matter how many layers the neural network contains if they all use linear activation functions the output is a linear combination of the input.

- The range of the output spans from (−∞ to +∞)(−∞ to +∞).
- Linear activation function is used at just one place i.e. output layer. Using linear activation
- across all layers makes the network's ability to learn complex patterns limited.



2. Non-Linear Activation Functions

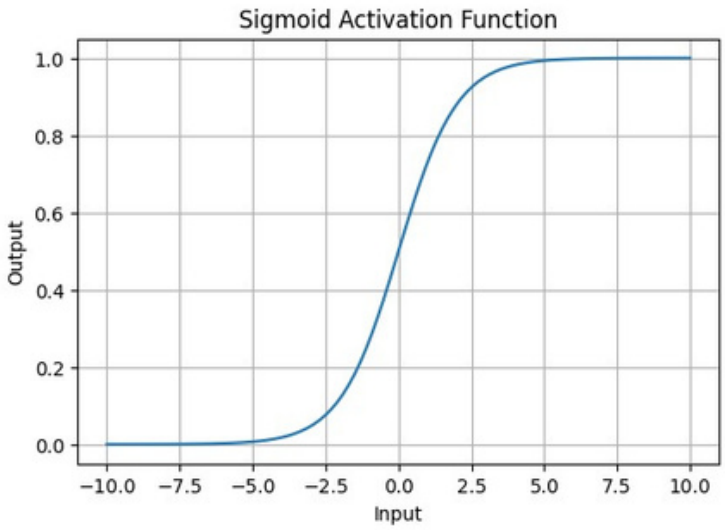
1. Sigmoid Function

The Sigmoid activation function is characterized by 'S' shape. Mathematically -

$$A = \frac{1}{1 + e^{-x}}$$

This formula ensures a smooth and continuous outputthat is essential for gradient-based optimization methods.

- It allows neural networks to handle and modelcomplex patterns that linear equations cannot.
- The output ranges between 0 and 1, hence usefulforbinary classification. The function
- exhibits a steep gradient whenxvalues are between -2 and 2. This sensitivity means that small changes in input xcancause significant changes in output y which is critical during the training process.

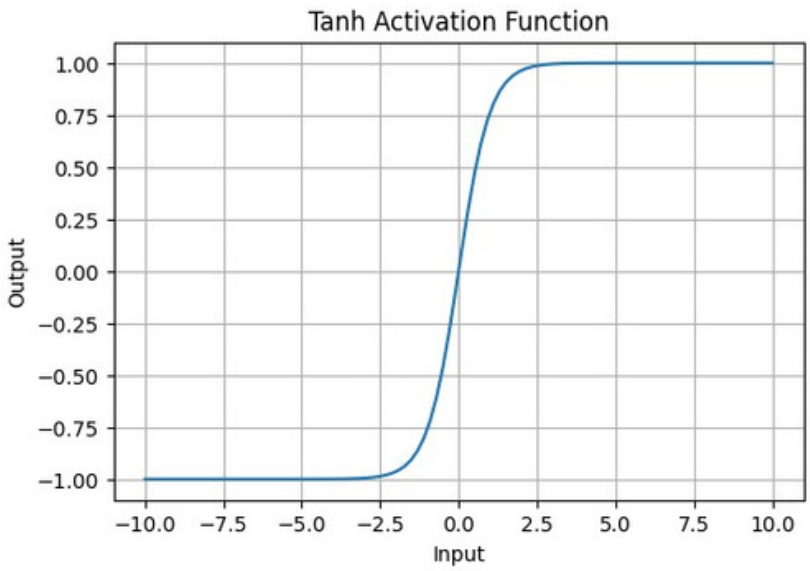


2. Tanh Activation Function

Tanh function (hyperbolic tangent function) is a shifted version of the sigmoid, allowing it to stretch across the y-axis. It is defined as:

$$f(x) = \tanh(x) = \frac{1 - e^{-2x}}{1 + e^{-2x}}$$

- Value Range: Outputs values from -1 to +1.
- Non-linear: Enables modeling of complex data patterns.
- Use in Hidden Layers: Commonly used in hidden layers due to its zero-centered output, facilitating easier learning for subsequent layers.



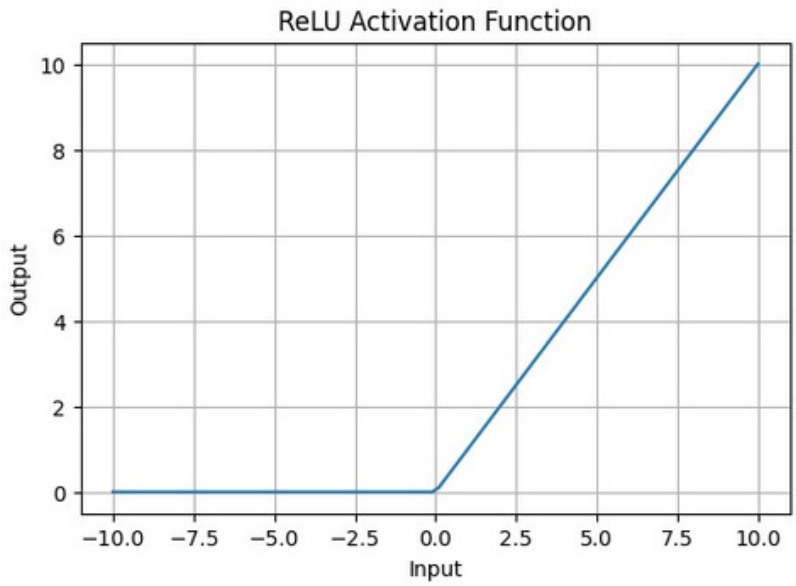
3. ReLU (Rectified Linear Unit) Function

ReLU function is defined by

$$A(x) = max(0, x)$$

this means that if the input x is positive, ReLU returns x, if theinput is negative, it returns 0.

- Value Range: [0,∞)[0,∞), meaning the function only outputs non-negative values. Nature
- : It is a non-linear activation function, allowing neural networks to learn complex patterns and making backpropagation more efficient.
- Advantage over other Activation: ReLU is less computationally expensive than tanh and sigmoid because it involves simpler mathematical operations. At a time only a few neurons are activated making the network sparse making it efficient and easy for computation.



Project Description and Methodology

1) Data Acquisition and Feature Selection:

The project begins with a structured student placement dataset containing academic, cognitive, and skill-based attributes. Each record corresponds to an individual student described by features such as:

- IQ
- Previous Semester Result
- CGPA
- Communication Skills
- Projects Completed
- Internship Experience

The target variable Placement is binary, where:

- 0 → Not Placed
- 1 → Placed
- Irrelevant or non-informative columns are removed to ensure that only meaningful features contribute to model learning.

2) Data Preprocessing and Normalization :

Before model training, the dataset undergoes preprocessing to improve learning stability and performance.

Feature Scaling (Standardization):

Since Support Vector Machines are distance-based models, all numerical features are scaled using StandardScaler, which transforms each feature as:

$$x' = \frac{x - \mu}{\sigma}$$

where

μ= mean of the feature

σ = standard deviation of the feature

This ensures:

Zero mean and unit variance

Equal contribution of all features

Faster convergence and stable margin estimations

The dataset is then split into **training and testing sets** to evaluate generalization on unseen data.

3*) Model Construction: Soft-Margin Linear Support Vector Machine:*

A Soft-Margin Linear SVM is employed to handle overlapping class distributions commonly present in real-world educational data.

$$f(x) = w^T x + b$$

where

- w is the weight vector
- b is the bias term

The optimization objective is:

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n} \xi_i$$

$$y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

Here:

- ξi are slack variables allowing misclassification
- C controls the trade-off between margin maximization and classification error

This formulation enables robust classification even when placed and non-placed student profiles overlap.

4) Model Training and Hyperparameter Tuning

The SVM model is trained on the scaled training dataset. The regularization parameter (C) is tuned across multiple values to analyze its effect on classification accuracy and recall.

Key benefits of this step:

- Prevents overfitting
- Controls model complexity
- Improves recall for the minority (placed) class

Stratified splitting ensures class balance during training and testing.

5) Model Evaluation and Validation:

Model performance is evaluated using standard classification metrics:

- Accuracy – overall correctness of predictions
- Precision – correctness of placement predictions
- Recall – ability to identify placed students
- F1-Score – balance between precision and recall

Additionally:

- Confusion Matrix is used for visual error analysis
- Cross-Validation ensures stability across multiple data splits

Decision scores obtained from the SVM indicate the confidence of placement prediction for individual students.

6) Prediction and Decision Interpretation:

For a new student input, the trained SVM computes a decision score:

- Positive score → Likely to be Placed
- Negative score → Likely to be Not Placed

This allows the system to act as a decision-support tool, assisting institutions in identifying students who may require additional academic or skill-based intervention.

CODE AND IMPLEMENTATION

Load data

```python
import pandas as pd
df = pd.read_csv("/content/College_placement.csv")
df.head()
```

Python

	Unnamed: 0	IQ	Prev_Sem_Result	CGPA	Academic_Performance	Extra_Curricular_Score	Communication_Skills	Projects_Completed	Internship_Experier
0	0	101	8.34	7.87	9	7	9	3	
1	1	120	7.97	7.76	1	9	9	2	
2	2	107	5.07	4.81	9	2	5	4	
3	3	106	7.72	8.11	8	1	9	2	
4	4	97	6.72	6.90	6	7	3	3	

Data splitting

```python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

Feature Scaling

```python
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Model Training.

```python
from sklearn.svm import SVC

svm = SVC(
    kernel="linear",
    C=1,
    class_weight="balanced"
)

svm.fit(X_train_scaled, y_train)
```

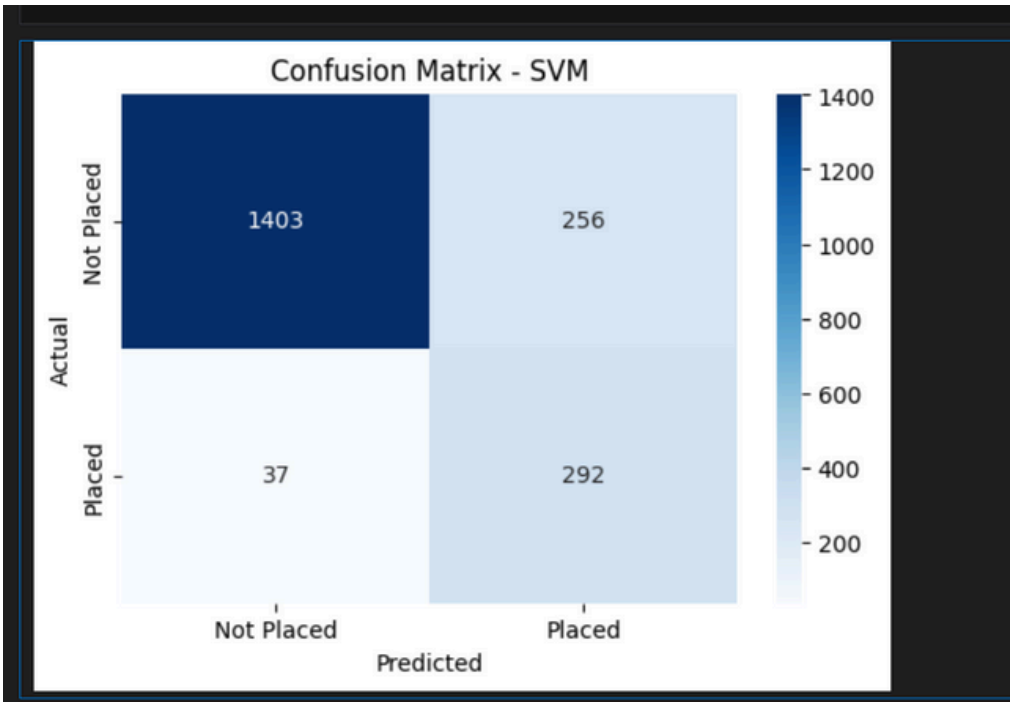
```
SVC
SVC(C=1, class_weight='balanced', kernel='linear')
```

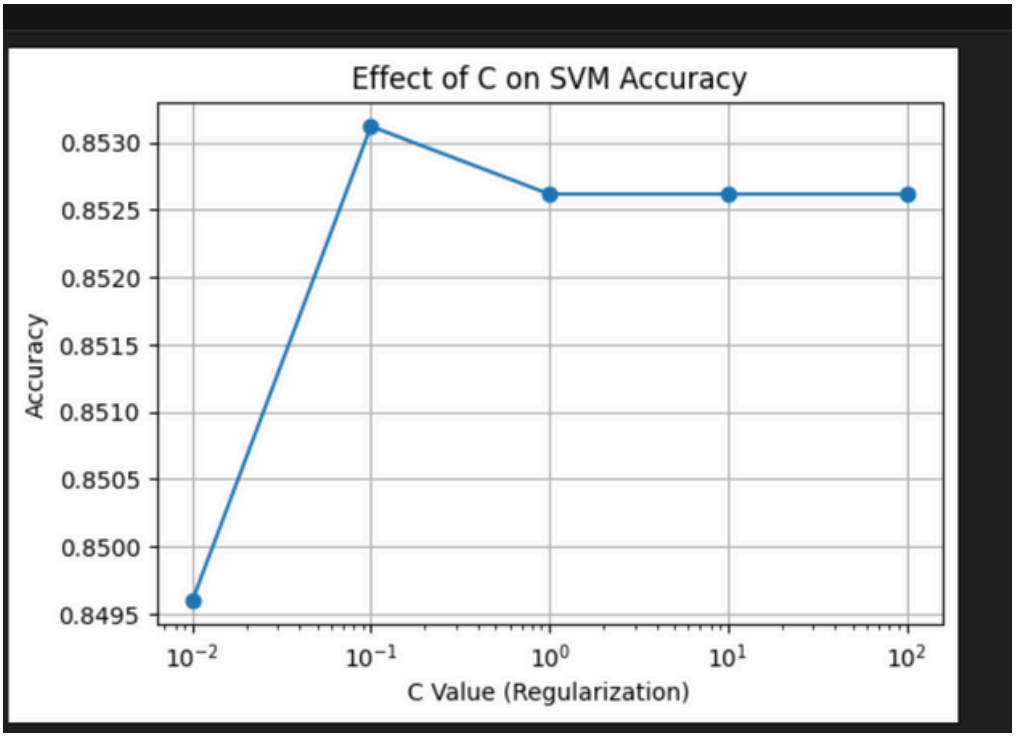
```python
y_pred = svm.predict(X_test_scaled)
```

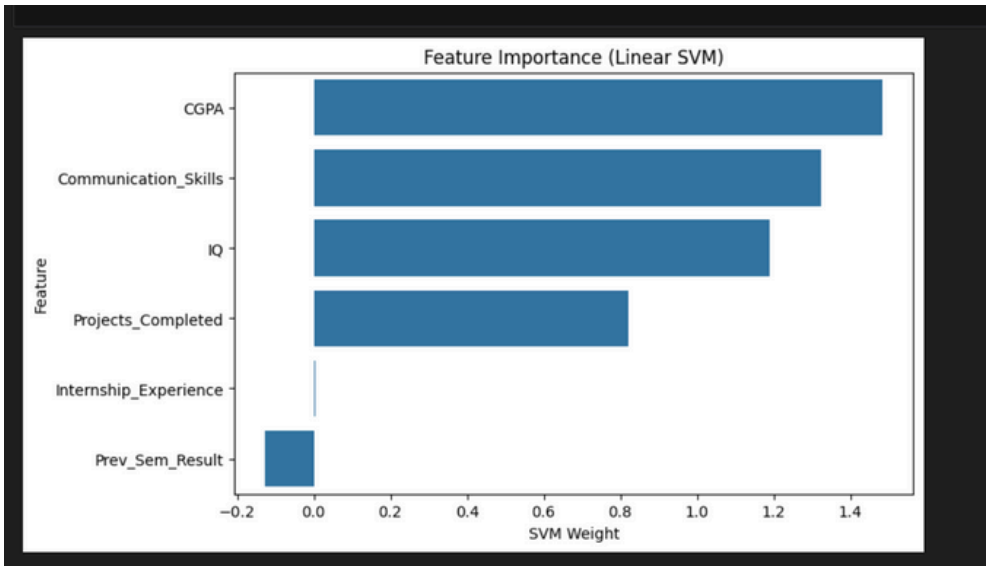
```python
from sklearn.metrics import accuracy_score, classification_report

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.85261569416499
              precision    recall  f1-score   support

           0       0.97      0.85      0.91      1659
           1       0.53      0.89      0.67       329

    accuracy                           0.85      1988
   macro avg       0.75      0.87      0.79      1988
weighted avg       0.90      0.85      0.87      1988
```

Visualizations :







Testing & validation:

```python
new_student = pd.DataFrame([{
    "IQ": 110,
    "Prev_Sem_Result": 7.8,
    "CGPA": 8.0,
    "Communication_Skills": 8,
    "Projects_Completed": 3,
    "Internship_Experience": 1
}])
new_student_scaled = scaler.transform(new_student)
prediction = svm.predict(new_student_scaled)

if prediction[0] == 1:
    print("✅ Student is likely to be PLACED")
else:
    print("❌ Student is likely NOT PLACED")
decision_score = svm.decision_function(new_student_scaled)
print("Decision Score:", decision_score)
from sklearn.model_selection import cross_val_score

scores = cross_val_score(
    svm, X_train_scaled, y_train, cv=5, scoring="accuracy"
)

print("Cross-validation accuracy:", scores)
print("Mean CV accuracy:", scores.mean())
```

```
✅ Student is likely to be PLACED
Decision Score: [1.21293257]
Cross-validation accuracy: [0.84789441 0.86666667 0.84716981 0.81886792 0.8490566 ]
Mean CV accuracy: 0.8459310824646498
```

(NOTE:The report includes all major implementation steps of the machine learning pipeline. Only key code snippets are presented for clarity, while the complete execution was carried out in a notebook environment.)

CONCLUSION :

This internship provided valuable hands-on experience in designing, implementing, and evaluating a machine learning–based student placement prediction system using real-world academic and skill-related data. The primary focus of the project was on applying a Soft-Margin Linear Support Vector Machine (SVM) to accurately classify students as placed or not placed, based on multiple performance indicators.

Practical Implementation – The project involved comprehensive data preprocessing, feature selection, and model training using Python and Scikit-learn. A structured machine learning pipeline was developed, including correlation-based feature analysis, feature scaling using standardization, and stratified train–test splitting to ensure balanced evaluation. The SVM model was successfully trained and tested on a student placement dataset, demonstrating its effectiveness in handling overlapping and non-linearly distributed real-world data.

Performance – The trained SVM model achieved strong and consistent classification performance, with high overall accuracy and stable results across multiple regularization (C) values. Cross-validation further confirmed the robustness of the model, yielding consistent accuracy across different data splits. The model showed particularly good recall for placed students, which is crucial in placement prediction scenarios where identifying eligible candidates is a priority.

Model Characteristics – The linear SVM classifier learns an optimal decision boundary by maximizing the margin between placed and non-placed students while allowing controlled misclassifications through soft-margin optimization. Feature scaling ensured equal contribution of all input variables, while class weighting effectively addressed class imbalance. The model's linear nature also enabled interpretability through feature weight analysis, providing insights into key factors influencing placement outcomes.

Deployment Insights – The developed model offers a strong balance between predictive accuracy, interpretability, and computational efficiency, making it suitable for deployment as a decision-support tool in academic institutions. It can be effectively used for early identification of students at risk of not being placed, enabling targeted academic and skill-based interventions. While the current implementation focuses on offline prediction, the model can be extended to web-based or interactive systems for real-time usage with minimal computational overhead.

Overall, this internship significantly strengthened practical understanding of machine learning workflows, supervised classification techniques, and model evaluation strategies within the domain of artificial intelligence and data science. The project established a solid foundation for future work, which may include exploring non-linear kernels, integrating additional behavioral and assessment features, and deploying the system as an intelligent, user-facing application to support placement-driven decision making